

G09/G16 MATLAB Toolbox

A parsing and visualisation toolbox for Gaussian 09/16 output files

Sebastiano Trusso

CNR – Istituto per i Processi Chimico-Fisici (IPCF), Messina, Italy

Version 1.0 — 2026

Contents

1	Introduction	3
2	Which .in keywords generate which .out section	4
3	File I/O utilities	6
3.1	G09_read_lines	6
3.2	G09_check_gaussian_match / G16_check_gaussian_match	6
4	Core data extraction	7
4.1	G09_route / G16_route	7
4.2	G09_structure / G16_structure	7
4.3	G09_energy / G16_energy	8
4.4	G09_charge_mult / G16_charge_mult	9
4.5	G09_convergence / G16_convergence	9
4.6	G09_dipole_polar / G16_dipole_polar	10
4.7	G09_nmodes / G16_nmodes	10
4.8	G09_spectra / G16_spectra	11
4.9	G_spectra_nm (shared, no G09_/G16_ prefix)	12
5	Atomic charges	14
5.1	G09_charges / G16_charges	14
5.2	G09_charges_fchk / G16_charges_fchk	15
6	Formatted checkpoint (.fchk) support	17
6.1	G09_fchk_read / G16_fchk_read	17
7	Molecule and normal-mode visualisation	19
7.1	G09_draw_molecule / G16_draw_molecule	19
7.2	G09_draw_mode / G16_draw_mode	20
7.3	G09_modeViewer / G16_modeViewer	21
7.4	G09_animate_mode / G16_animate_mode	21
8	Orbital energy analysis	24
8.1	G09_orbital_energies / G16_orbital_energies	24
8.2	G09_draw_orbital / G16_draw_orbital	25
9	Response properties (G16 only)	26

9.1	G16_hyperpolar	26
9.2	G16_tddft	26
10	Structural analysis utilities	28
10.1	G09_get_bond_length / G16_get_bond_length	28
10.2	G09_nbo_bonds / G16_nbo_bonds	28
10.3	G09_gaussian_version / G16_gaussian_version	29
10.4	G09_restart / G16_restart	29
10.5	G09_available_data / G16_available_data	30
10.6	G_read_input (shared, no G09_/G16_ prefix)	30
10.7	G09_list / G16_list	31
11	One-call data extraction	33
11.1	G09_read_all / G16_read_all	33
11.2	G09_batch_read_all / G16_batch_read_all	34
12	Report generation	35
12.1	G09_write_report / G16_write_report	35
12.2	Function reference	36
12.3	Selected signatures	38
12.4	Notes on the port	39
13	Worked examples	43
13.1	example_1.m — Structure, vibrational mode, and charges	43
13.2	example_2.m — Infrared and Raman spectra	44
13.3	example_3.m — Energetics	45
13.4	example_4.m — Recovering and restyling graphics handles	47
14	Publication and licensing	49
14.1	Declaration of Generative AI and AI-assisted technologies in the writing process	49
14.2	Note on the Use of AI-Assisted Development Tools	49
14.3	Licence	49
14.4	Citation	49

1 Introduction

This manual documents the **G09/G16 Toolbox**, a set of MATLAB functions for parsing, analysing, and visualising Gaussian 09 and Gaussian 16 output files (`.out/.log`) and formatted checkpoint files (`.fchk`). The toolbox covers:

- extraction of molecular geometry, energies, thermochemistry, dipole moment, polarisability, and hyperpolarisability;
- vibrational analysis: normal modes, IR/Raman spectra, and interactive mode visualisation;
- molecular-orbital energy diagrams and TD-DFT excited-state analysis;
- 3D molecular structure rendering (CPK ball-and-stick models) with atomic charge and dipole overlays;
- utility functions for bond-length tables, Gaussian version detection, restart-file generation, and toolbox self-documentation.

Function names follow the `G09_xxx/G16_xxx` convention, indicating which Gaussian version's output format the function targets. Where the two versions behave identically, this manual documents them in a single combined subsection (`G09_xxx / G16_xxx`); where behaviour differs, the differences are called out explicitly.

Every parsing function that reads a `.out/.log` file accepts an optional `'Lines'` Name-Value parameter: a cell array of pre-read file lines. This lets `G09_read_all/G16_read_all` read the file from disk once and reuse the same lines across every sub-parser, instead of each function re-reading and re-splitting the file independently. Passing `'Lines'` is entirely optional — omitting it falls back to reading the file normally.

Wrong-toolbox warning. Because Gaussian 09 and Gaussian 16 output formats differ, using the wrong toolbox on a file (e.g. `G16_energy` on a Gaussian 09 file) can silently misparse it rather than raising an obvious error — for example, `G16_dipole_polar` on a Gaussian 09 file returns `NaN` for the polarisability instead of an error. To catch this, every function that reads a file from disk checks the Gaussian-version citation line (`"Gaussian NN, Revision X.YY,"`) it just read and prints a non-blocking `warning(...)` if `NN` does not match the toolbox being used, suggesting the other one instead. The check runs only once per actual disk read (see Section 3.2), so `G09_read_all/G16_read_all` produce a single warning, not one per sub-parser.

2 Which .in keywords generate which .out section

Each extraction function reads one specific section of the .out/ .log file, identified by a fixed header string or line pattern. Most of those sections only appear if the corresponding Gaussian route keyword was present in the .in/.gjf job that produced the file — otherwise the function raises an error (or, for optional response-property fields, returns NaN/empty rather than failing). The table below maps each function to the .out section it reads and the .in keyword(s) that generate it. Function names omit the G09_/G16_ prefix (identical for both versions unless noted).

Function	.out section read	.in keyword	Notes
structure	"Standard orientation" / "Input orientation"	(default)	Always printed. nosymm/nosym suppresses "Standard orientation", leaving only "Input orientation"
energy (SCF)	"SCF Done:"	(default)	Printed by any HF/DFT energy evaluation (single point, opt, freq, ...)
energy (thermo-chemistry)	"Zero-point correction=", "Sum of electronic and ... Energies="	freq	ZPE, H, G, S
charges (Mulliken)	"Mulliken charges:"	(default)	
charges (APT)	"APT charges:"	freq	Printed automatically with any frequency job (dipole-derivative charges) — no separate pop=apt needed
dipole_polar (dipole)	"Dipole moment (...)"	(default)	
dipole_polar (static Al-pha)	"Exact polarizability:"	polar	
dipole_polar (dynamic Al-pha)	"Alpha(-w,w) frequency N"	polar cphf=rdfreq	Requires an explicit frequency list (a.u.) after the route/title/charge-multiplicity block in the .in file
nmodes	"and normal coordinates:"	freq	
spectra (IR)	"Harmonic frequencies ... IR intensities"	freq	
spectra (Raman)	"Harmonic frequencies ... Raman scattering"	freq=raman	
orbital_energies	"Alpha occ./virt. eigenvalues" (+ Beta ... for unrestricted)	(default)	Full eigenvalue list is printed unconditionally, not gated by pop=full
convergence	"Item Value Threshold Converged?"	opt	

Function	.out section read	.in keyword	Notes
charge_mult	charge/multiplicity echo near the top of the file	(default)	Mirrors the .in charge/-multiplicity line
route	route-section echo near the top of the file	(default)	
get_bond_length	derived from the structure section	(default)	Purely geometric (covalent radii); no dedicated Gaussian section
nbo_bonds	"Natural Bond Orbitals (Summary)"	pop=nbo	See Section 10.2
gaussian_version	"Gaussian NN, Revision X.YY," citation line	(default)	
hyperpolar (vibrational Beta, G16 only)	"Diagonal vibrational hyperpolarizability:"	polar freq	Present even without cphf=rdfreq
hyperpolar (electronic Beta, G16 only)	"First dipole hyperpolarizability, Beta ...", "Beta(0;0,0)", "Beta(-w;w,0)"	polar field=... cphf=rdfreq	Needs the field/frequency list in the .in file; polar alone gives only the vibrational Beta above
tddft (G16 only)	"Excited State N: ..."	td (e.g. td=(nstates=N))	
restart	reuses the structure/route/charge_mult sections above	(any completed step)	

.fchk functions. `fchk_read` and `charges_fchk` (provided in both G09/ and G16/) do not read .out/.log sections at all — they read a formatted checkpoint file, produced by running `formchk` on the binary .chk file written by the job's `%chk=... link0` command. No specific route keyword is required beyond `%chk=...` itself.

Evidence, not assumption. The keyword requirements above were verified directly against real Gaussian output files rather than inferred from general documentation — e.g. APT charges and the full orbital-eigenvalue list turned out to be printed unconditionally (no `pop=apt/pop=full` needed), and the vibrational vs. electronic hyperpolarisability distinction was confirmed by comparing a file run with plain `polar` against one run with `field=... cphf=rdfreq`, only the latter containing the full electronic Beta tensor.

3 File I/O utilities

3.1 G09_read_lines

```
lines = G09_read_lines(filename)
```

Low-level reader used internally by every other **G09_XXX** parser. Reads a Gaussian 09 output file and returns a cell array of lines, with line endings stripped. Transparently handles CRLF line endings (as produced by Windows G09W builds) and ISO-8859-1 (Latin-1) encoding.

G16 parsers read files directly with the platform default encoding rather than calling a separate **G16_read_lines** helper; the 'Lines' parameter interface is identical across both toolboxes.

3.2 G09_check_gaussian_match / G16_check_gaussian_match

```
ok = G09_check_gaussian_match(lines, filename)
ok = G16_check_gaussian_match(lines, filename)
```

Internal helper, called automatically by every function that reads a **.out/.log** file from disk (not part of the typical user-facing API, but documented here for completeness). Scans the already-read **lines** (no extra disk I/O) for the "Gaussian NN, Revision X.YY," citation line printed near the top of every output file. If found and NN does not match the toolbox (9 for **G09_***, 16 for **G16_***), prints a non-blocking warning suggesting the other toolbox; otherwise does nothing. Returns **true** if no mismatch was detected (or the version is unknown), **false** if a warning was issued.

Because this check lives inside each function's own "read the file fresh from disk" branch, it only fires when a file is actually opened — if a function receives pre-read **lines** via the 'Lines' parameter (e.g. every sub-parser called by **G09_read_all/G16_read_all**), it skips both the disk read and the check, so a single call to **read_all** produces exactly one warning, not one per sub-parser.

4 Core data extraction

4.1 G09_route / G16_route

```
route = G09_route(filename)
route = G16_route(filename)
route = G16_route(filename, 'Lines', lines)
```

Extracts the complete route section (the # line(s) between the two ---- separators) as a single string.

Option	Default	Description
'Lines'	{}	Pre-read file lines (Section 3.1); read normally if omitted

```
r = G09_route('indaco.log');
% '# opt freq=raman b3lyp/6-311++g(d,p) nosymm geom=connectivity field
=x+50'
```

Both `G09_route/G16_route` and `G09_charge_mult/ G16_charge_mult` now accept the 'Lines' option, consistent with the rest of the toolbox.

4.2 G09_structure / G16_structure

```
mol = G09_structure(filename)
mol = G09_structure(filename, 'step', 'last')
mol = G09_structure(filename, 'step', 'first')
mol = G09_structure(filename, 'step', N)

mol = G16_structure(filename)
mol = G16_structure(filename, 'orientation', 'standard')
mol = G16_structure(filename, 'orientation', 'input')
```

Extracts the molecular geometry from a Gaussian output file.

Difference: Gaussian 09 writes only "Input orientation:" blocks (never "Standard orientation:"), so `G09_structure` has no 'orientation' option. Gaussian 16 can write either, so `G16_structure` exposes 'orientation' = 'standard' (default, falls back to input if absent) | 'input' | 'auto'.

Option	Default	Description
'step'	'last'	Which geometry block to extract: 'first', 'last', or an integer index
'orientation'	'auto' (G16 only)	'standard' 'input' 'auto'
'Lines'	{}	Pre-read file lines; read normally if omitted

Field	Type	Description
.symbols	cell	Atomic symbols
.xyz	double	Coordinates in Å (X Y Z)
.Z	int	Atomic numbers
.Natoms	int	Number of atoms
.step	int	Index of the extracted step (1-based)
.n_steps	int	Total geometry blocks found in the file
.orientation	char	'Input orientation' (always, G09) or 'standard'/'input' (G16)
.filename	char	Source file name

```
mol = G09_structure('indaco.log');
mol = G16_structure('calc.out', 'orientation', 'input');
```

4.3 G09_energy / G16_energy

```
en = G09_energy(filename)
en = G09_energy(filename, 'step', 'last')
en = G09_energy(filename, 'step', N)
```

Extracts SCF energy and thermochemistry data.

Field	Type	Description
.SCF	double	SCF energy (Hartree)
.method	char	Method string, e.g. 'RB3LYP'
.ZPE_corr	double	Zero-point correction (Hartree)
.U_corr	double	Thermal correction to energy (Hartree)
.H_corr	double	Thermal correction to enthalpy (Hartree)
.G_corr	double	Thermal correction to Gibbs free energy (Hartree)
.E0	double	SCF + ZPE (Hartree)
.U	double	SCF + U_corr (Hartree, G16 only)
.H	double	SCF + H_corr (Hartree)
.G	double	SCF + G_corr (Hartree)
.S_JmolK	double	Entropy S (J mol ⁻¹ K ⁻¹ , G09)
.ZPE_kJ	double	Zero-point energy (kJ/mol)
.T, .P	double	Temperature (K), pressure (atm)
.has_thermo	logical	false for opt-only jobs (no freq); *_corr, E0, U, H, G, T, P are NaN in that case
.G_kJ, .H_kJ	double	G , H in kJ/mol (G09)
.filename	char	

Option	Default	Description
'step'	'last'	Which SCF block to extract: 'last', 'first', or integer N
'Lines'	{}	Pre-read file lines

4.4 G09_charge_mult / G16_charge_mult

```
[charge, mult] = G09_charge_mult(filename)
[charge, mult] = G16_charge_mult(filename)
[charge, mult] = G16_charge_mult(filename, 'Lines', lines)
```

Extracts molecular charge and spin multiplicity from the "Charge = ... Multiplicity = ..." line. Format identical across G09 and G16.

```
[q, m] = G09_charge_mult('indaco.log'); % q = 0, m = 1
```

4.5 G09_convergence / G16_convergence

```
cv = G09_convergence(filename)
cv = G09_convergence(filename, 'plot', true)

cv = G16_convergence(filename)
cv = G16_convergence(filename, 'plot', true)
```

Extracts geometry-optimisation convergence criteria (Max/RMS Force, Max/RMS Displacement) at every optimisation step, together with their thresholds and whether/when the job converged. Block format identical across G09 and G16.

Option	Default	Description
'plot'	false	Plot the four convergence series against their thresholds

Field	Type	Description
.MaxForce, .RMSForce	vector	Max / RMS force per step (a.u.)
.MaxDisp, .RMSDisp	vector	Max / RMS displacement per step (a.u.)
.thr_MaxForce, .thr_RMSForce, .thr_MaxDisp, .thr_RMSDisp	double	Corresponding convergence thresholds
.converged	logical	Whether all four criteria were satisfied
.conv_step	int/NaN	Step at which convergence was reached
.Nsteps	int	Number of optimisation steps found
.filename	char	

```
cv = G16_convergence('V_E00t.out', 'plot', true);
cv.converged
cv.conv_step
```

4.6 G09_dipole_polar / G16_dipole_polar

```
dp = G09_dipole_polar(filename)
dp = G09_dipole_polar(filename, 'units', 'Debye')

dp = G16_dipole_polar(filename)
dp = G16_dipole_polar(filename, 'units', 'SI')
```

Extracts the dipole moment and polarisability tensor.

Differences between G09 and G16: G09 prints "Exact polarizability:" as six values on a single line (upper triangle, row order), rather than G16's formatted Alpha(0;0)/Alpha(-w;w) blocks. G09 has no dynamic (laser-frequency-dependent) polarisability unless requested via Polar=OptRot or similar keywords, and additionally prints a "Diagonal vibrational polarizability:" line (3 values, returned in .alpha_vib). An approximate CPHF tensor, if printed by G09 ("Approx polarizability:"), is returned in .alpha_approx. G16 instead reports full dynamic polarisability entries per laser frequency (.alpha_dyn) and modal derivatives .dmu_dQ/.dalpha_dQ (related to IR intensities and Raman activities; for exact scaled values use G16_spectra instead).

Option	Default	Description
'units'	'au'	'au' 'Debye' (dipole only, G09) / 'Debye' (full, G16) 'SI'
'Lines'	{}	Pre-read file lines

Field	Description
.mu_x, .mu_y, .mu_z, .mu_tot, .mu_units	Dipole components, magnitude, unit string
.alpha_iso, .alpha_aniso	Isotropic polarisability, anisotropy
.alpha_tensor, .alpha_units	3×3 static tensor (a.u.), unit string
.alpha_vib, .has_alpha_vib	Diagonal vibrational polarisability (a.u.), logical (G09)
.alpha_approx	CPHF approximate tensor (a.u.), NaN if absent (G09)
.alpha_dyn, .N_dyn	Struct array of dynamic polarisability entries (per laser frequency), count (G16)
.dmu_dQ, .dalpha_dQ, .has_deriv	Dipole/polarisability derivatives w.r.t. normal modes, logical (G16)
.filename	

4.7 G09_nmodes / G16_nmodes

```
nm = G09_nmodes(filename)
nm = G09_nmodes(filename, 'modes', [5 10 20])

nm = G16_nmodes(filename)
```

```
nm = G16_nmodes(filename, 'section', 'last')
```

Extracts normal-mode displacement vectors.

G09 writes only one *Harmonic frequencies* section per file (even for opt+freq jobs), so G09_nmodes has no 'section' option — it is implicit. G16 may contain more than one such section (e.g. a preliminary and a final one) and defaults to the last.

Option	Default	Description
'modes'	[]	Restrict output to specific mode indices
'section'	'last' (G16 only)	'last' 'first'
'Lines'	{}	Pre-read file lines

Field	Description
.freq,	.IR, Frequencies (cm^{-1}), IR intensities (KM/Mole), Raman activities
.Raman	($\text{\AA}^4/\text{AMU}$, [] if absent)
.redmass,	Reduced masses (AMU), force constants ($\text{mDyne/\AA}$)
.frconst	
.symmetry	Symmetry labels
.disp	$[N_{\text{atoms}} \times 3 \times N_{\text{modes}}]$ Cartesian displacement vectors
.Nmodes,	
.Natoms,	
.has_Raman,	
.filename	

4.8 G09_spectra / G16_spectra

```
sp = G09_spectra(filename)
sp = G09_spectra(filename, Name, Value, ...)

sp = G16_spectra(filename)
sp = G16_spectra(filename, 'section', 'last')
```

Extracts IR/Raman line spectra and generates Lorentzian-broadened continuous spectra.

G09 writes only one Harmonic-frequencies section per file, so G09_spectra needs no 'section' parameter; G16_spectra exposes it to select which section to use when more than one is present.

Option	Default	Description
'FWHM'	10	Lorentzian FWHM (cm^{-1})
'xmin'	0	Lower wavenumber limit (cm^{-1})
'xmax'	4000	Upper wavenumber limit (cm^{-1})
'dx'	1	Grid step (cm^{-1})
'normalize'	false	Normalise continua to maximum = 1
'plot'	false	Generate figure
'section'	'last' (G16 only)	'last' 'first'
'Lines'	{}	Pre-read file lines

Field	Description
.freq, .Raman	.IR, Frequencies (cm^{-1}), IR intensities, Raman activities ([] if absent)
.has_Raman, .Nmodes	Logical, number of modes
.x, .Raman_cont	.IR_cont, Wavenumber grid, broadened IR/Raman continua
.FWHM, .filename	

```
sp = G16_spectra('calc.out', 'FWHM', 15, 'xmin', 400, 'plot', true);
```

4.9 G_spectra_nm (shared, no G09_/G16_ prefix)

```
sp = G_spectra_nm(nm)
sp = G_spectra_nm(nm, Name, Value, ...)
```

Builds the same Lorentzian-broadened IR/Raman continuum spectra as `G09_spectra`/`G16_spectra`, but directly from an already-parsed `nm` (normal modes) struct instead of reading a `.log/.out` file. Accepts the `nm` struct returned by `G09_nmodes`/`G16_nmodes`, or the `.nm` sub-struct returned by `G09_fchk_read`/`G16_fchk_read` (Section 6.1) — all four share the same field layout (`.freq`, `.IR`, `.Raman`, `.has_Raman`, `.Nmodes`, `.filename`), so a single function works with any of them regardless of Gaussian version or data source.

This fills a gap left by `G09_fchk_read`/`G16_fchk_read`: their output struct's `.nm` field has frequencies and IR/Raman intensities, but — unlike `G09_spectra`/`G16_spectra` — does not itself build the broadened continuum spectrum. `G_spectra_nm` does exactly that, directly from `data.nm`, with no `.out/.log` file needed at all.

Option	Default	Description
'FWHM'	10	Lorentzian FWHM (cm^{-1})
'xmin'	0	Lower wavenumber limit (cm^{-1})
'xmax'	4000	Upper wavenumber limit (cm^{-1})
'dx'	1	Grid step (cm^{-1})
'normalize'	false	Normalise continua to maximum = 1
'plot'	false	Generate figure

Output fields are identical to G09_spectra/G16_spectra's (Section 4.8 above): .freq, .IR, .Raman, .has_Raman, .Nmodes, .x, .IR_cont, .Raman_cont, .FWHM, .filename.

```
data = G16_fchk_read('molecule.fchk');
sp    = G_spectra_nm(data.nm, 'FWHM', 15, 'plot', true);    % from a .
      fchk, no .out needed

nm = G16_nmodes('molecule.out');
sp = G_spectra_nm(nm, 'xmin', 400, 'normalize', true);      % from an
      already-read .out
```

5 Atomic charges

5.1 G09_charges / G16_charges

```
ch = G09_charges(filename)
ch = G09_charges(filename, 'type', 'APT')
ch = G09_charges(filename, 'mode', 'heavy')
ch = G09_charges(filename, 'plot', True, 'threshold', 0.05)
```

Extracts Mulliken or APT atomic charges and optionally renders them on the 3D structure, with an optional dipole-moment arrow overlay.

Robustness: handles all known Mulliken label variants across Gaussian revisions/OSes (e.g. "Mulliken atomic charges:" on G09 Rev. A.02/Windows and some Linux builds; "Mulliken charges:" on G09 Rev. B.01/C.01/D.01 and on all G16 revisions), by matching any line containing the charge-type keyword and "charges:", excluding "Sum of" and H-summed lines. The exact label found is reported in `ch.label`.

Option	Default	Description
'type'	'Mulliken'	'Mulliken' 'APT'
'mode'	'atom'	'atom' 'heavy' (H charges summed onto heavy atoms)
'plot'	True	Render labels on the 3D structure
'AtomScale'	0.35	CPK sphere scale
'BondTol'	1.30	Bond detection tolerance
'BondList'	[]	Passed straight through to <code>draw_molecule</code> : explicit atom-index pairs (optionally with a pre-computed bond order), bypassing <code>BondTol</code> — see the note below
'FontSize'	8	Charge-label font size
'ColorScale'	'RdBu'	'RdBu' (blue=neg/red=pos) 'none'
'threshold'	0	Hide labels with $ q < \text{threshold}$
'ShowDipole'	False	Overlay the total dipole moment vector
'DipoleOrigin'	'negcharge'	Arrow anchor: 'negcharge' (centroid of negative charges) 'poscharge' 'centroid' [x y z]
'DipoleScale'	1.0	Arrow length, Å per Debye
'DipoleColor'	[0 0.6 0]	Arrow colour
'DipoleLineWidth'	2.5	Arrow line width
'ShowDipoleLabel'	True	Annotate the arrow with $ \mu $
'DipoleFontSize'	11	Dipole label font size
'DipoleUnits'	'Debye'	Units for $ \mu $: 'Debye' 'au'
'Lines'	{}	Pre-read file lines

Field	Description
.symbols, .charges	Atomic symbols, per-atom charges (e)
.charges_H	H-summed charges on heavy atoms
.sum_q	Sum of all charges (≈ 0 for neutral molecules)
.type, .label	'Mulliken'/'APT', exact header label found
.Natoms	

Rendering NBO-derived bond orders. 'BondList' accepts the same [Nbonds x 3] Atom1, Atom2, BondOrder matrix that draw_molecule does (Section 10.2), so the charge overlay renders on top of the real NBO-derived bond order instead of the geometric estimate:

```
bt = G16_nbo_bonds('molecule_nbo.out');
bond_list = [bt.Atom1, bt.Atom2, double(bt.BondOrder)];
G16_charges('molecule_nbo.out', 'BondList', bond_list, 'ShowDipole', true);
```

5.2 G09_charges_fchk / G16_charges_fchk

```
ch_out = G09_charges_fchk(mol, ch)
ch_out = G09_charges_fchk(mol, ch, Name, Value, ...)

ch_out = G16_charges_fchk(mol, ch)
```

Visualises atomic charges directly from a G09_fchk_read/ G16_fchk_read struct, without requiring the corresponding .log/.out file. Mirrors the interface of G09_charges/ G16_charges exactly, so the two can be used interchangeably once data.mol and data.ch are available.

Option	Default	Description
'mode'	'atom'	'atom' 'heavy' (requires ch.charges_H non-empty — .fchk files have no native H-summed data; use G09_charges/G16_charges on the .log for that)
'plot'	true	
'AtomScale'	0.35	
'BondTol'	1.30	
'FontSize'	8	
'ColorScale'	'RdBu'	'RdBu' 'none'
'threshold'	0	

```
data = G16_fchk_read('3typ.fchk');
ch = G16_charges_fchk(data.mol, data.ch); %
    default
ch = G16_charges_fchk(data.mol, data.ch, 'plot', false, 'threshold',
    0.05);
ch = G16_charges_fchk(data.mol, data.ch, 'ColorScale', 'none'); %
    drop-in for G16_charges
```

Requires `mol` with fields `.symbols`, `.xyz`, `.Natoms`, `.filename`, and `ch` with fields `.charges`, `.symbols`, `.type`, `.Natoms` (`.charges_H` optional).

Since the `.fchk` format is identical for G09 and G16, `G09_charges_fchk` and `G16_charges_fchk` are functionally identical (the latter simply calls `G16_draw_molecule` internally instead of `G09_draw_molecule`) — both are provided so that either toolbox folder alone is enough.

6 Formatted checkpoint (.fchk) support

6.1 G09_fchk_read / G16_fchk_read

```
data = G09_fchk_read(filename)
data = G09_fchk_read(filename, 'verbose', false)

data = G16_fchk_read(filename)
```

Reads a Gaussian 09/16 formatted checkpoint file (**.fchk**), generated with **formchk jobname.chk jobname.fchk**. Uses a single generic section parser (no hard-coded line counts) and is compatible with both G09 and G16 **.fchk** files.

Supersedes the old **.fck/newzmat**-format reader: reads the modern **.fchk** ASCII format, returns SI-ready fields (XYZ in Å, dipole in Debye), and needs no external helper functions. The formatted-checkpoint layout does not depend on the Gaussian version, so **G09_fchk_read** and **G16_fchk_read** are identical and each works on **.fchk** files produced by *either* G09 or G16 — both are provided purely so that either toolbox folder alone (G09/ or G16/) is enough to read any **.fchk** file, without needing the other one on the path too.

Option	Default	Description
'verbose'	true	Print progress messages while parsing

Output struct data (grouped by section):

Field	Description
.title, .method, .basis	First line, method (e.g. 'RB3LYP'), basis set (e.g. 'def2SVPP')
.Nat, .charge, .mult	Number of atoms, molecular charge, spin multiplicity
.Nelec, .Nalpha, .Nbeta	Total, alpha, beta electron counts
.Nbasis, .Nbasis_indep	Basis functions, independent basis functions
.symbols, .AN, .masses	Atomic symbols, atomic numbers, real atomic masses (AMU)
.xyz, .xyz_bohr	Coordinates (Å), coordinates (Bohr)
.SCF_energy, .total_energy, .virial_ratio	SCF energy, total energy (Hartree), virial ratio (≈ 2)
.alpha_orb_ energies, .beta_orb_energies	MO energies (Hartree); beta is [] if RHF
.alpha_MO_coeff	Alpha MO coefficients (column-major)
.mulliken_charges	Mulliken charges (e)
.HOMO_idx, .HOMO_eV, .LUMO_eV, .gap_eV	Frontier-orbital summary
.gradient, .rms_force	Cartesian gradient (Hartree/Bohr), RMS force

7 Molecule and normal-mode visualisation

7.1 G09_draw_molecule / G16_draw_molecule

```
G09_draw_molecule(mol)
G09_draw_molecule(mol, 'Name', Value, ...)
```

Renders a 3D CPK ball-and-stick model from a mol struct (as returned by `G09_structure/G16_structure`), with bonds detected automatically from a covalent-radius criterion and drawn as 1, 2, or 3 parallel lines according to an estimated bond order.

Option	Default	Description
'AtomScale'	0.35	Sphere radius as a fraction of the covalent radius
'BondTol'	1.30	Bond tolerance: bonded if $d < (r_1 + r_2) \times \text{BondTol}$
'ShowLabels'	true	Show atom labels ("C1", "H2", ...)
'ShowLegend'	true	Show the element-colour legend
'Title'	filename	Figure title
'BgColor'	[0.95 0.95 0.95]	Axes background colour
'Ax'	(new figure)	Draw into an existing axes handle
'ShowAxes'	false	Draw a small Cartesian X/Y/Z reference triad, anchored at the lower-left-front corner of the plotted molecule
'AxesLength'	[]	Length of the X/Y/Z arrows, in Å ([] = auto, 20% of the molecule's bounding-box diagonal)
'BondList'	[]	[$N_{\text{bonds}} \times 2$] or [$N_{\text{bonds}} \times 3$] explicit atom-index pairs (optionally with a 3rd column giving a pre-computed bond order 1/2/3) to draw as bonds, bypassing the BondTol distance criterion ([] = auto-detect). Used by <code>G09_animate_mode/G16_animate_mode</code> to keep a fixed bond topology (and, with the 3rd column, a fixed bond order) across all frames of an animation

```
mol = G09_structure('zeatin.out');
G09_draw_molecule(mol);
G09_draw_molecule(mol, 'ShowLabels', false, 'AtomScale', 0.4);
G09_draw_molecule(mol, 'ShowAxes', true);
```

Bond order (single/double/triple). For C-C and C-O pairs, bond order is estimated purely from bond length (any other element pair, including C-N, is always drawn as a single bond) and rendered as 1/2/3 parallel lines in the usual chemical-drawing convention — a geometric estimate, not Gaussian's own bond-order analysis (e.g. Wiberg/NBO indices). For an actual bond order derived from Gaussian's own NBO analysis, see `G09_nbo_bonds/G16_nbo_bonds` (Section 10.2). The C-C double/single threshold is deliberately set to 1.36 Å rather than the generic reference-length midpoint (≈ 1.44 Å): symmetric aromatic rings have essentially equal C-C bond lengths (≈ 1.39 – 1.40 Å, with no real single/double alternation to recover from geometry alone), so the generic midpoint would classify every ring bond as double. With the adjusted threshold, aromatic rings are instead rendered as all-single — a more honest depiction than an arbitrary alternating pattern, since the true bond order is ≈ 1.5 all the way around the ring.

Rendering NBO-derived bond orders. `nbo_bonds`'s output table can be fed straight into `draw_molecule`'s `'BondList'` parameter (Section 10.2), replacing the geometric estimate above with the real bond order from Gaussian's own NBO analysis:

```
mol = G16_structure('molecule_nbo.out');
bt  = G16_nbo_bonds('molecule_nbo.out');    % requires 'pop=nbo' in
      the source job

bond_list = [bt.Atom1, bt.Atom2, double(bt.BondOrder)];
G16_draw_molecule(mol, 'BondList', bond_list);
```

Why C-N is excluded. C-N was classified by length in an earlier version of the toolbox, using the same kind of threshold as C-C/C-O. Unlike C-C, a single threshold cannot separate aromatic C-N (e.g. pyridine rings, ≈ 1.34 Å) from a genuine C=N: on asymmetric pyridine-like rings this produced one ring C-N bond rendered double and its chemically-equivalent neighbour rendered single, an inconsistent and misleading picture. C-N is therefore always drawn as a single bond, regardless of length.

Bug fixed (2026-08-05): lighting not following interactive rotation. The plot supports interactive rotation via `rotate3d` (drag with the mouse). CAMLIGHT only positions a light relative to the camera at the moment it is called — MATLAB's own documentation is explicit about this: "In order for a light created with CAMLIGHT to stay in a constant position relative to the camera, CAMLIGHT must be called whenever the camera is moved." ROTATE3D does not do this on its own, so previously the two lights stayed fixed in the axes' 3-D world frame while the molecule rotated underneath them: after a large enough rotation (e.g. 180 degrees) the CPK spheres ended up lit from behind instead of from the viewer's side, losing the shading gradient that gives them their 3-D appearance and making them look flat/uniformly lit. Fixed by keeping a handle to both lights and assigning `rotate3d`'s `ActionPostCallback` property (not an event — assigning it again simply replaces the previous callback, so repeated calls into the same figure, e.g. one `draw_molecule` call per `animate_mode` frame, do not accumulate callbacks) to reposition both lights via CAMLIGHT after every interactive rotation.

No differences between the G09 and G16 versions beyond the input struct's origin.

7.2 G09_draw_mode / G16_draw_mode

```
G09_draw_mode(mol, nm, mode_idx)
G09_draw_mode(mol, nm, mode_idx, 'Name', Value, ...)
```

Renders a single vibrational normal mode as displacement arrows over the CPK structure.

Option	Default	Description
'Scale'	1.5	Arrow length scale
'ArrowColor'	[1 0.4 0.1]	Arrow colour
'AtomScale'	0.35	CPK sphere scale factor
'BondTol'	1.30	Bond detection tolerance
'ShowLabels'	false	Show atom index labels
'FlipSign'	false	Invert all arrow directions

Why 'FlipSign' exists. Normal-mode eigenvectors have an arbitrary overall sign — a mode and its 180°-phase-shifted twin are physically identical. If the arrows point opposite to GaussView's rendering for a given mode, set 'FlipSign', true to flip them. This is purely cosmetic and has no effect on frequencies or intensities.

7.3 G09_modeViewer / G16_modeViewer

```
G09_modeViewer(filename)
G09_modeViewer(filename, 'Name', Value, ...)
```

Interactive selector GUI for browsing vibrational modes. Reads the structure and all normal modes via `G09_structure/G09_nmodes`, then opens a window listing every mode (index, frequency, symmetry). Selecting an entry calls `G09_draw_mode` to render that mode in a new figure window; previously drawn mode figures stay open, so different modes (or render settings) can be compared side by side.

Window controls:

- *Order by:* sort the mode list by mode number (default), IR intensity, or Raman intensity (descending; only offered when Raman data is present). The current selection is preserved across reordering.
- A text box to set a custom title on the current mode figure.
- A control for every `G09_draw_mode` option (`Scale`, `ArrowColor`, `AtomScale`, `BondTol`, `ShowLabels`, `FlipSign`); changing any of them immediately redraws the selected mode in a new figure. Arrow colour can be set from a named preset (Orange/Red/Blue/Green/Black/ Magenta) or a custom colour.
- A *Save figure...* button exporting the target mode figure to JPEG, EPS, or PDF (EPS/PDF as true vector graphics).
- An *Animate mode (MP4)...* button exporting an MP4 animation of the currently selected mode via `G09_animate_mode/ G16_animate_mode`, using the current `Scale/AtomScale/ BondTol/ShowLabels/FlipSign` settings and, automatically, the camera orientation of the currently displayed mode figure (a manual rotation carries over into the animation).

Name-Value arguments passed to `G09_modeViewer` pre-populate the corresponding option control(s), e.g. `G09_modeViewer('file.log', 'Scale', 2, 'ShowLabels', true)`.

```
G09_modeViewer('V_E00t.out');
G16_modeViewer('INDACO_E025_ppDP.LOG', 'Scale', 2, 'ShowLabels', true)
;
```

See also `G09_structure/G16_structure`, `G09_nmodes/G16_nmodes`, `G09_draw_mode/G16_draw_mode`, `G09_animate_mode/G16_animate_mode`.

7.4 G09_animate_mode / G16_animate_mode

```
outfile = G09_animate_mode(mol, nm, mode_idx)
outfile = G09_animate_mode(mol, nm, mode_idx, 'Name', Value, ...)

outfile = G16_animate_mode(mol, nm, mode_idx)
```

Exports an MP4 animation of a single vibrational mode: the molecule oscillates along the mode's displacement vector (equilibrium $\pm$ amplitude), in the style of GaussView's mode animations, and the result is saved via `VideoWriter` ('MPEG-4' profile). A figure window opens and animates

live while the video is rendered; avoid interacting with it until the function returns.

Option	Default	Description
'Filename'	[] (auto)	Output path; default <source>.mode<N>.mp4, .mp4 appended if missing
'Scale'	1.5	Displacement amplitude scale, same meaning as G09_draw_mode/G16_draw_mode's 'Scale'
'FlipSign'	false	Invert the displacement direction
'AtomScale'	0.35	See G09_draw_molecule/G16_draw_molecule
'BondTol'	1.30	See G09_draw_molecule/G16_draw_molecule; also used to compute the fixed bond list (see below)
'ShowLabels'	false	See G09_draw_molecule/G16_draw_molecule
'FramesPerCycle'	30	Frames per oscillation period
'NCycles'	2	Number of periods rendered
'FPS'	20	Video frame rate
'View'	[]	[azimuth elevation] in degrees, the starting camera orientation ([] = MATLAB's default 3D view, azimuth= -37.5, elevation= 30). Pass the output of MATLAB's own view(ax) to match an already-rotated figure
'ShowWaitbar'	true	Print rendering progress to the command window
'Crop'	true	Tightly crop every frame around the molecule instead of writing the whole (mostly blank) figure window (see below)
'CropMargin'	12	Blank margin kept around the molecule when 'Crop' is true, in pixels
'ShowTitle'	false	Draw the filename/mode/frequency title text in every frame. Left off by default so the title's own bounding box does not force a taller crop than the molecule itself needs

```
mol = G16_structure('V_E00t.out');
nm = G16_nmodes('V_E00t.out');
G16_animate_mode(mol, nm, 70, 'Scale', 2, 'FPS', 25);
```

Fixed bond list and bond order. Bonds (and their estimated order, 1/2/3) are computed once from the *equilibrium* geometry (via G09_get_bond_length/G16_get_bond_length, using 'BondTol' as the tolerance) and passed to G09_draw_molecule/G16_draw_molecule through its 'BondList' parameter (atom-index pairs plus a 3rd bond-order column), instead of being re-detected from instantaneous inter-atomic distances on every frame. Without this, bonds would flicker in and out — and double/triple bonds would flicker to/from single — as oscillating atoms cross the relevant distance thresholds.

No graphical waitbar. Rendering progress is printed to the command window rather than shown with MATLAB's `waitbar`: on some MATLAB versions, `waitbar` itself was found to error intermittently across many rapid updates ("Not enough input arguments" inside its own `title()` call).

Axes handling. Each frame's axes are deleted and recreated (rather than cleared with `cla`) before redrawing: in this 3D-plus-lighting configuration, `cla(ax)` was found to leave stale graphics objects behind, causing the exported video to show all frames superimposed instead of one at a time.

Frame cropping. Without `'Crop'`, every frame is the full figure window (default MATLAB figure size), leaving most of it blank around the molecule. The crop rectangle is computed *once*, from the two oscillation extremes (phase +1 and -1, which bound the whole animation), by finding the tight bounding box of non-background pixels and padding it by `'CropMargin'`; the same rectangle is then applied to every frame. Because the axis limits are already fixed across the whole animation (see above), this rectangle stays valid throughout the video without ever clipping an atom at any phase, typically reducing the frame area by 70–80%.

No differences between the G09 and G16 versions beyond the input structs' origin.

See also `G09_draw_molecule/G16_draw_molecule`, `G09_draw_mode/G16_draw_mode`, `G09_modeViewer/G16_modeViewer`, `G09_get_bond_length/G16_get_bond_length`.

8 Orbital energy analysis

8.1 G09_orbital_energies / G16_orbital_energies

```
oe = G09_orbital_energies(filename)
oe = G09_orbital_energies(filename, 'step', 'last')
oe = G09_orbital_energies(filename, 'step', N)
```

Extracts molecular orbital energies (HOMO/LUMO and the full occupied/virtual spectrum) from a Gaussian 09/16 output file, by parsing the "Alpha occ. eigenvalues --" / "Alpha virt. eigenvalues --" lines printed by Gaussian's population analysis (and the corresponding "Beta" lines for open-shell calculations). These blocks repeat once per SCF calculation in the file (e.g. once per optimisation step); the 'step' option selects which block to report, consistently with G09_energy/G09_structure.

Option	Default	Description
'step'	'last'	Which eigenvalue block to use: 'first', 'last', or an integer index

Output struct oe (energies in Hartree unless noted):

Field	Type	Description
.alpha_occ	vector	Occupied alpha orbital energies
.alpha_virt	vector	Virtual alpha orbital energies
.beta_occ	vector	Occupied beta orbital energies ([] if closed-shell)
.beta_virt	vector	Virtual beta orbital energies ([] if closed-shell)
.has_beta	logical	true for open-shell (UHF/UKS) calculations
.HOMO / .LUMO	double	Highest occupied / lowest unoccupied orbital energy
.gap	double	LUMO -- HOMO
.HOMO_alpha / .LUMO_alpha	double	Alpha HOMO/LUMO (== .HOMO/.LUMO for closed-shell)
.HOMO_beta / .LUMO_beta	double	Beta HOMO/LUMO ([] if closed-shell)
.HOMO_eV / .LUMO_eV / .gap_eV	double	Same three quantities, in eV
.step	int	Block index actually used
.Nsteps	int	Number of eigenvalue blocks found in the file
.filename	char	Source file name

```
oe = G09_orbital_energies('V_E00t.out');
fprintf('HOMO-LUMO gap = %.3f eV\n', oe.gap_eV);
```

For open-shell (UHF/UKS) calculations, HOMO and LUMO are computed as $\max(\text{HOMO}_\alpha, \text{HOMO}_\beta)$ and $\min(\text{LUMO}_\alpha, \text{LUMO}_\beta)$ respectively, so .gap always refers to the smallest alpha/beta HOMO-LUMO gap. No differences between the G09 and G16 versions beyond the parsed file's origin.

8.2 G09_draw_orbital / G16_draw_orbital

```
G09_draw_orbital(oe)
G09_draw_orbital(oe, 'Name', Value, ...)
```

Draws a molecular-orbital energy-level diagram from the struct returned by `G09_orbital_energies/G16_orbital_energies` showing a user-selected number of occupied/virtual levels around the frontier orbitals and highlighting the HOMO–LUMO transition with an arrow labelled with the gap value.

Option	Default	Description
'NLevels'	5	Number of occupied/virtual levels shown on each side of the frontier orbitals
'Units'	'eV'	Energy units: 'eV' or 'Hartree'
'Ax'	(new figure)	Draw into an existing axes handle
'OccColor'	[0.25 0.25 0.70]	Colour for non-frontier occupied levels
'VirtColor'	[0.70 0.30 0.25]	Colour for non-frontier virtual levels
'HOMOColor'	[0.00 0.45 0.85]	Highlight colour for the HOMO
'LUMOCOLOR'	[0.90 0.35 0.00]	Highlight colour for the LUMO
'ArrowColor'	[0.15 0.60 0.15]	HOMO→LUMO transition arrow colour
'ShowLabels'	true	Annotate each level with its energy value
'ShowSpins'	true	Draw a paired-electron arrow glyph ($\uparrow\downarrow$) on occupied levels
'FontSize'	8	Font size for level energy-value labels
'FrontierFontSize'	13	Font size for the HOMO/LUMO tags and the gap annotation
'ArrowHeadSize'	0.2	HOMO–LUMO arrow head size, as a fraction of the arrow length (passed to <code>quiver</code>)
'Title'	filename or 'Orbital Energy Diagram'	Figure title

```
oe = G09_orbital_energies('a1.out');
G09_draw_orbital(oe)
G09_draw_orbital(oe, 'NLevels', 8, 'Units', 'Hartree')
```

Requires the `oe` struct returned by `G09_orbital_energies/ G16_orbital_energies` and errors if `alpha_occ/alpha_virt` are missing or empty. No differences between the G09 and G16 versions beyond the input struct's origin.

9 Response properties (G16 only)

9.1 G16_hyperpolar

```
hp = G16_hyperpolar(filename)
hp = G16_hyperpolar(filename, 'units', 'au')      % default
hp = G16_hyperpolar(filename, 'units', 'esu')    % 10-30 esu
hp = G16_hyperpolar(filename, 'units', 'SI')     % 10-50 C3 m3 J-2
```

Extracts the dipole hyperpolarisability β (static, dynamic, and vibrational contributions) from a Gaussian 16 output file.

Option	Default	Description
'units'	'au'	'au' 'esu' (10 ⁻³⁰ esu) 'SI' (10 ⁻⁵⁰ C ³ m ³ J ⁻²)

Field	Description
.beta0.par_z,	Parallel / perpendicular component of static $\beta(0;0,0)$ along z
.beta0.perp_z	
.beta0.vec_x/y/z,	Vector components and magnitude $ \beta_{\text{vec}} $
.beta0.beta_vec	
.beta0.tensor	All Cartesian tensor components ($xxx, xyx, \dots, zzz$)
.beta_dyn	Struct array, one entry per laser frequency: .lambda_nm, .par_z, .perp_z, .vec_x/y/z, .beta_vec, .tensor
.N_dyn	Number of dynamic (laser) frequencies found
.beta_vib,	Diagonal vibrational hyperpolarisability, logical
.has_vib	
.filename	

```
hp = G16_hyperpolar('V_E00t.out');
hp.beta0.beta_vec          % static beta modulus, a.u.
hp.beta_dyn(1).lambda_nm  % wavelength of the first dynamic entry (nm)
hp.beta_dyn(1).beta_vec   % |beta| at that frequency
```

9.2 G16_tddft

```
td = G16_tddft(filename)
td = G16_tddft(filename, 'nstates', N)      % first N states only
td = G16_tddft(filename, 'plot', true)      % also generate UV-Vis plot
td = G16_tddft(filename, 'FWHM_eV', 0.3)    % broadening FWHM (eV)
```

Extracts TD-DFT excited states and generates a Gaussian-broadened UV-Vis spectrum.

Option	Default	Description
'nstates'	Inf	Load only the first N excited states
'plot'	false	Generate the UV-Vis plot
'FWHM_eV'	0.30	Gaussian broadening FWHM (eV)

Field	Description
.n, .mult	State index; multiplicity label (e.g. 'Singlet-A', 'Triplet-B')
.eV, .nm, .f	Excitation energy (eV), wavelength (nm), oscillator strength
.S2, .has_S2	$\langle S^2 \rangle$ expectation value (0 for pure singlets), logical
.trans	Cell array, one $[N_{\text{contrib}} \times 3]$ matrix per state: columns [MO_from, MO_to, coeff] (MO_to < 0 denotes a de-excitation)
.Nstates	Total number of excited states loaded
.x_nm, .eps_cont	Wavelength grid (100–1000 nm), Gaussian-broadened UV-Vis spectrum (arb. units)
.FWHM_eV, .filename	

```

td = G16_tddft('violacein_td.out');
plot(td.x_nm, td.eps_cont)      % UV-Vis spectrum
stem(td.nm, td.f)              % transition stem plot
td.trans{2}                    % transitions contributing to the 2nd
    state

```

No G09 equivalent is provided: TD-DFT excited-state analysis and hyperpolarisability response properties in this toolbox are currently supported only for Gaussian 16 output files.

10 Structural analysis utilities

10.1 G09_get_bond_length / G16_get_bond_length

```
bondTable = G09_get_bond_length(mol)
bondTable = G09_get_bond_length(mol, 'Name', Value, ...)
```

Builds the bond-length table of a molecule from a `mol` struct (fields `.symbols`, `.xyz`), detecting bonds via a covalent-radius criterion, and returns a MATLAB table.

Option	Default	Description
'Tolerance'	1.15	Multiplicative factor on the sum of covalent radii for the bonding criterion
'IncludeH'	true	Include bonds involving hydrogen atoms
'SortBy'	'distance'	Sort by 'distance' (ascending) or 'atom' (atom index)
'SaveAs'	''	Path (.xlsx or .csv) to save the table to; not saved if omitted

Output columns: Atom1, Sym1, Atom2, Sym2, Distance Ang.

```
T = G09_get_bond_length(mol, 'SortBy', 'atom');
T = G16_get_bond_length(mol, 'IncludeH', false, 'SaveAs', 'bonds.xlsx');
```

The bonding criterion is purely geometric (covalent radii), not derived from Gaussian's own connectivity/topology matrix — suitable for quick structural analysis, not as the "official" G09/G16 connectivity.

10.2 G09_nbo_bonds / G16_nbo_bonds

```
bt = G09_nbo_bonds(filename)
bt = G09_nbo_bonds(filename, 'Name', Value, ...)
```

Determines bond order (single/double/triple) from an actual Gaussian NBO (Natural Bond Orbital) analysis, as opposed to `get_bond_length`'s geometric covalent-radius criterion or `draw_molecule`'s bond-length heuristic. Reads the "Natural Bond Orbitals (Summary)" table and counts, for every atom pair, how many bonding (BD) natural bond orbitals NBO assigned to it: one BD orbital means a single bond (σ only), two mean a double bond ($\sigma + \pi$), three mean a triple bond ($\sigma + 2\pi$). Antibonding (BD*) orbitals are excluded.

Option	Default	Description
'section'	'last'	Which "Natural Bond Orbitals (Summary)" block to read, for files with more than one (e.g. multi-step opt+freq+polar jobs that repeat the NBO analysis at each step); also accepts 'first'
'Lines'	{}	Pre-read cell array of file lines, to skip re-reading the file

Output columns: Atom1, Sym1, Atom2, Sym2, BondOrder, Occupancy (summed NBO occupancy, ≈ 2 per BD orbital).

```
bt = G09_nbo_bonds('CH4_NBO.LOG');
```

Requires the source file to have been computed with the `pop=nbo` Gaussian keyword (or similar, e.g. `pop=(nbo,savenbo)`); without it, the output file simply does not contain NBO data and the function raises an error — bond order cannot be recovered after the fact from geometry/energy alone. This reads the "Natural Bond Orbitals (Summary)" table that `pop=nbo` always prints, not the separate "Wiberg bond index matrix" (which additionally requires `pop=(nbo,bndidx)` and is not parsed by this function).

10.3 G09_gaussian_version / G16_gaussian_version

```
gv = G09_gaussian_version(filename)
gv = G16_gaussian_version(filename)
```

Detects which Gaussian version/revision produced a `.out/.log/.fchk` file.

For `.out/.log` files, reads the "Gaussian NN, Revision X.YY," citation line printed near the top of the file (works for any NN: 09, 16, ...). `.fchk` files do not store this information at all; in that case the function looks for a sibling `.log/.out` file with the same base name in the same folder and reads the version from there. If no sibling file is found, `.major/.revision/.full` are returned empty and a warning is issued.

Field	Description
<code>.major</code>	Gaussian major version, e.g. 9, 16 ([] if unknown)
<code>.revision</code>	Revision string, e.g. 'A.02', 'C.01' ('' if unknown)
<code>.full</code>	Citation line as printed, e.g. 'Gaussian 09, Revision A.02'
<code>.source</code>	'out/log' 'fchk-sibling:<path>' 'unknown'
<code>.filename</code>	

```
gv = G09_gaussian_version('a1.out');          % gv.major = 16, gv.
      revision = 'C.01'
gv = G16_gaussian_version('molecule.fchk'); % falls back to molecule.
      out/.log if present
```

10.4 G09_restart / G16_restart

```
gjf_file = G09_restart(filename)
gjf_file = G09_restart(filename, 'Name', Value, ...)

gjf_file = G16_restart(filename)
gjf_file = G16_restart(filename, 'Name', Value, ...)
```

Generates a Gaussian 09/16 input file (`.gjf`) to restart a calculation from a geometry found in an existing `.log/.out` file. `G16_restart` reads geometry via `G16_structure` (so it follows that function's Standard/Input orientation handling); otherwise the two functions are identical.

Option	Default	Description
'step'	'last'	'last' 'first' integer N — which geometry step to restart from
'output'	<base>_restarted.out	Output .gjf path
'extra_route'	''	Additional keywords appended to the route section
'nproc'	0	%nprocshared override (0 = keep/omit)
'mem'	''	%mem override ('' = keep/omit)

If the source output file has no %chk line (common in Gaussian 09 Windows output), the checkpoint name is derived from the source file name instead. The generated .gjf is valid for restarting under either Gaussian 09 or Gaussian 16, and can be read back with G_read_input.

Returns the path of the generated .gjf file.

10.5 G09_available_data / G16_available_data

```
T = G09_available_data(filename)
T = G16_available_data(filename)
```

Reads the route section (see G09_route/G16_route) and checks it against the keyword requirements documented in Section 2 ("Which .in keywords generate which .out section"), so you can tell up front which data will actually be present in a file — e.g. a TD-DFT-only single point job (td=(singlets,nstates=100), no freq) has no vibrational normal modes, no IR/Raman spectra, and no thermochemistry (ZPE/H/G/S), so calling nmodes/spectra on it raises an error and energy's *_corr/E0/U/H/ G/T/P/S fields come back NaN — none of that is a bug, it is simply not in the file. This function lets you check that in one call, rather than discovering it one error/NaN at a time.

Field	Description
.Function	Toolbox function/quantity name
.Available	logical — true if the route contains the keyword(s) needed
.Requires	Human-readable keyword requirement (e.g. 'freq', 'polar + cphf=rdfreq')
.Notes	Short clarifying note

```
G16_available_data('V_E00_R_24_TD.out');
T = G16_available_data('molecule.out');
T(~T.Available, :) % see what's NOT available in this file
```

This predicts availability from the route keywords alone; it does not itself read the data sections, so an incomplete/crashed job can still show a keyword as present without the corresponding output actually having been printed (the extraction function will still raise its own clear error in that case). G09_available_data omits the hyperpolar/tddft rows entirely, since those functions do not exist in the G09 toolbox.

10.6 G_read_input (shared, no G09_/G16_ prefix)

```
ginp = G_read_input(filename)
```

Reads a Gaussian *input* file (.gjf, .com, or .in) and extracts the link0 commands, route section, title, charge/multiplicity, and starting Cartesian geometry.

The Gaussian input file format does not differ between Gaussian 09 and Gaussian 16, so this is the toolbox's only function without a G09_/G16_ prefix pair: an identical copy is kept in both the G09/ and G16/ folders, so it is available regardless of which toolbox you have on the MATLAB path. Only Cartesian-coordinate geometry blocks are supported (not Z-matrix input).

Field	Type	Description
.symbols	cell	Atomic symbols
.xyz	double	Starting coordinates in Å (X Y Z)
.Natoms	int	Number of atoms
.charge	int	Total molecular charge
.mult	int	Spin multiplicity
.title	char	Title/comment line(s), joined with a space
.route	char	Full route section, single line
.method	char	Method parsed from the route (e.g. 'b3lyp'), '' if not found
.basis	char	Basis set parsed from the route (e.g. '6-311+g(d,p)'), '' if not found
.chk	char	%chk link0 value, '' if absent
.mem	char	%mem link0 value, '' if absent
.nproc	char	%nprocshared/%nproc link0 value, '' if absent
.filename	char	Source file path

ginp has the same .symbols/.xyz/.Natoms/ .filename fields as the struct returned by G09_structure/G16_structure, so it can be passed directly to G09_draw_molecule/G16_draw_molecule or G09_get_bond_length/G16_get_bond_length without any conversion.

```
ginp = G_read_input('molecule.gjf');
G16_draw_molecule(ginp, 'ShowAxes', true);
fprintf('%s/%s, charge %d, mult %d\n', ginp.method, ginp.basis, ...
        ginp.charge, ginp.mult);
```

10.7 G09_list / G16_list

```
G09_list()
T = G09_list()

G16_list()
T = G16_list()
```

Lists every G09_*.m/G16_*.m function found in the toolbox folder, printing the function name

and its H1 description line (sorted alphabetically). Also returns the same information as a **table** with columns `.Name`, `.Description`, `.File`.

```
G09_list();  
T = G16_list();  
T(contains(T.Description, 'dipole', 'IgnoreCase', true), :)
```

11 One-call data extraction

11.1 G09_read_all / G16_read_all

```
T = G09_read_all(filename)
T = G16_read_all(filename)
```

Runs the full set of G09_XXX/G16_XXX parsers on a single output file and collects the results into one struct T. The file is read from disk once and the resulting lines are passed to every sub-parser via their 'Lines' option, instead of each sub-parser independently re-reading and re-splitting the file.

Field	Source function	Description
.charge	G09/G16_charges	Mulliken and APT atomic charges (plotting disabled)
.energy	G09/G16_energy	SCF energy and thermochemistry
.structure	G09/G16_structure	Optimized molecular structure
.dipolar	G09/G16_dipole_polarizability	Dipole moment and polarisability
.nmodes	G09/G16_nmodes	IR/Raman vibrational normal modes; omitted (with a non-blocking warning) if the file has no frequency calculation — check with <code>isfield(T, 'nmodes')</code>
.spectra	G09/G16_spectra	Frequencies, IR/Raman intensities, simulated spectra; omitted under the same condition as <code>.nmodes</code>
.route	G16_route (G16 only)	Gaussian route section details
.chargemol.charge	G16_charge_mult	Total molecular charge, spin multiplicity
.chargemol.mol	(G16 only)	

```
T = G16_read_all('zeatin.out');
disp(T.energy)
G16_draw_molecule(T.structure);
```

Known discrepancy: the current `G09_read_all` does *not* populate `.route` or `.chargemol`, unlike `G16_read_all`, which retains both. If parity with `G16_read_all` is required, add

```
T.route = G09_route(filename, 'Lines', lines);
[c, m] = G09_charge_mult(filename, 'Lines', lines);
T.chargemol.charge = c;
T.chargemol.mol = m;
```

to `G09_read_all.m`.

Bug fixed (2026-08-05): `G09_read_all/G16_read_all` (and the Python `g16_read_all`) used to call `nmodes/spectra` unconditionally, so a job with no frequency calculation at all (e.g. a TD-DFT-only single point job, `td=(singlets,nstates=100)` with no `freq` keyword) crashed the *entire* call, even though every other field (charges, energy, structure, dipole, route, charge/multiplicity) would have extracted perfectly fine. This directly contradicted `G09_write_report.m/G16_write_report.m` (and their Python counterpart), which already used `isfield/getattr` checks expecting `.nmodes/.spectra` to be *option-*

ally absent. Fixed by wrapping just those two calls in `try/catch`: on failure the field is omitted entirely (not left as an empty placeholder) and a single non-blocking `warning(...)` explains why, restoring the graceful-degradation behaviour the rest of the toolbox already assumed.

11.2 G09_batch_read_all / G16_batch_read_all

```
T = G09_batch_read_all(folder)
T = G09_batch_read_all(folder, 'Recursive', true)
T = G09_batch_read_all(folder, 'WriteReports', true)
T = G09_batch_read_all(folder, 'SaveAs', 'summary.xlsx')

T = G16_batch_read_all(folder)
```

Scans folder for `.log/.out` files (case-insensitive), runs `G09_read_all/G16_read_all` (plus `G09_orbital_energies/G16_orbital_energies`, for the HOMO-LUMO gap) on each, and returns one row per file — useful for screening many calculations at once (conformers, scans, batches of jobs) without calling `read_all` file-by-file.

A file that fails to parse (e.g. an incomplete/crashed job, or a file from the other Gaussian version run through the wrong toolbox) does *not* stop the batch: its row is filled with NaN and the caught error message is recorded in the `.Status` column instead.

Option	Default	Description
'Recursive'	false	Also scan subfolders
'WriteReports'	false	Write a <code>G09_write_report/G16_write_report .txt</code> next to each source file
'SaveAs'	''	Path to save the summary table (<code>.csv</code> or <code>.xlsx</code> , inferred from the extension)

Field	Description
.File	Source file path
.Natoms	Atom count (from <code>.structure.Natoms</code>)
.SCF_Hartree	SCF energy
.E0_Hartree	SCF + ZPE
.G_kJmol	Gibbs free energy, kJ/mol
.mu_tot, .mu_units	Dipole magnitude and its units
.HOMO_eV, .LUMO_eV, .Gap_eV	Frontier orbital energies and gap, eV
.Status	'ok', or the caught error message if parsing failed

```
T = G16_batch_read_all('results/', 'WriteReports', true);
T(T.Gap_eV == min(T.Gap_eV), :) % smallest HOMO-LUMO gap
```

12 Report generation

12.1 G09_write_report / G16_write_report

```
outfile = G09_write_report(T)
outfile = G09_write_report(T, outfile)

outfile = G16_write_report(T)
outfile = G16_write_report(T, outfile)
```

Writes a human-readable text report from a `G09_read_all`/`G16_read_all` struct `T`, formatting a summary of every field present in `T` into a plain `.txt` file. If `outfile` is omitted, the file is named after the source Gaussian file (e.g. `'zeatin.out' → 'zeatin_report.txt'`) in the current folder.

Argument	Default	Description
<code>T</code>	(required)	Struct returned by <code>G09_read_all</code> / <code>G16_read_all</code>
<code>outfile</code>	<code><source>_report.txt</code>	Output <code>.txt</code> path

Returns the path written to. Sections are included only if the corresponding field is present in `T`, in this order: route, charge/multiplicity, molecular structure (full Cartesian coordinate table), energetics (with thermochemistry if available), dipole moment and polarisability (including the dynamic polarisability table, if present), atomic charges (full per-atom table, sum of charges, dipole-from-charges overlay if computed), vibrational normal modes (frequency/IR/Raman/reduced mass/symmetry table), and a summary of the simulated IR/Raman spectra.

The full broadened-spectrum continuum arrays (`T.spectra.x`, `.IR_cont`, `.Raman_cont`) are deliberately *not* dumped into the report — only the grid range, FWHM, and whether Raman data is present are summarised. Access those arrays directly from `T.spectra` for plotting or export.

```
T = G16_read_all('zeatin.out');
G16_write_report(T);                % -> zeatin_report.txt
G16_write_report(T, 'zeatin_summary.txt'); % explicit output path
```

No differences between the G09 and G16 versions beyond the input struct's origin and the report header ("Gaussian 09"/"Gaussian 16 Calculation Report").

See also `G09_read_all`/`G16_read_all`.

`G16parser` is a Python 3 port of the G16 MATLAB toolbox, covering the same parsing, checkpoint-analysis, and static-visualisation functionality described in the previous sections. It is installable as a standard editable package and exposes one `g16_xxx` function per MATLAB `G16_xxx` counterpart, each returning a `Struct` object (attribute access, e.g. `mol.xyz`, `ch.dipole.Debye`) instead of a MATLAB struct.

```
cd G16parser
pip install -e .
```

```
import G16parser as g16

mol = g16.g16_structure('molecule.out')
g16.g16_draw_molecule(mol, show_axes=True)

ch = g16.g16_charges('molecule.out', show_dipole=True)
```

```
oe = g16.g16_orbital_energies('molecule.out')
g16.g16_draw_orbital(oe)

T = g16.g16_read_all('molecule.out')    # everything in one call,
    single file read
```

Requirements: Python ≥ 3.9 , numpy, pandas, matplotlib (installed automatically via `pip install -e .`). No Gaussian installation or compiled dependency is needed.

12.2 Function reference

Every function mirrors its MATLAB `G16_xxx` counterpart in name, parameters (in `snake_case`), defaults, and returned fields — see the corresponding MATLAB subsection earlier in this manual for full parameter/field documentation. All extraction functions accept an optional `lines=` parameter (pre-read file lines) mirroring the MATLAB `'Lines'` option; `g16_read_all` uses this internally to read the file exactly once.

Function	Description
<code>g16_read_input</code>	Reads a Gaussian <i>input</i> file (<code>.gjf/.com/.in</code>): link0 commands, route, title, charge/multiplicity, and starting geometry — output <code>Struct</code> is compatible with <code>g16_draw_molecule/g16_get_bond_length</code>
<code>g16_structure</code>	Molecular geometry (symbols, xyz as <code>np.ndarray</code> , atom count)
<code>g16_energy</code>	SCF energy and thermochemistry
<code>g16_convergence</code>	Geometry-optimisation convergence criteria per step (<code>plot=True</code> for a matplotlib figure)
<code>g16_charges</code>	Mulliken/APT atomic charges, optional dipole overlay and 3D plot
<code>g16_dipole_polar</code>	Dipole moment and (static/dynamic) polarisability
<code>g16_nmodes</code>	Vibrational normal-mode displacement vectors
<code>g16_spectra</code>	IR/Raman spectra (stick + Lorentzian-broadened continuum)
<code>g16_spectra_nm</code>	Same Lorentzian-broadened IR/Raman spectra as <code>g16_spectra</code> , built directly from an already-parsed <code>nm</code> struct (from <code>g16_nmodes</code> , or the <code>.nm</code> field of <code>g16_fchk_read</code>) — no <code>.log/.out</code> file needed
<code>g16_orbital_energies</code>	HOMO/LUMO and the full occupied/virtual MO spectrum
<code>g16_charge_mult</code>	Molecular charge and spin multiplicity
<code>g16_route</code>	Route section string
<code>g16_available_data</code>	Lists which toolbox quantities are expected to be extractable from a file, by checking its route keywords (e.g. “no <code>freq</code> keyword → no <code>nmodes/spectra/thermochemistry</code> ”) — catches this before an error or a NaN
<code>g16_get_bond_length</code>	Bond-length table (pandas DataFrame) from covalent radii
<code>g16_nbo_bonds</code>	Bond order (single/double/triple) from an actual Gaussian NBO analysis (<code>pop=nbo</code>), by counting bonding (BD) natural bond orbitals per atom pair
<code>g16_fchk_read</code>	Reads a Gaussian formatted checkpoint (<code>.fchk</code>) file — works on both G09 and G16 <code>.fchk</code> files, since the format does not depend on the Gaussian version
<code>g16_charges_fchk</code>	Visualises charges from a <code>g16_fchk_read</code> struct, without needing the corresponding <code>.log/.out</code> file
<code>g16_gaussian_version</code>	Detects the Gaussian version/revision (works on <code>.fchk</code> via a sibling <code>.log/.out</code>)
<code>g16_hyperpolar</code>	Dipole hyperpolarisability (β)
<code>g16_tddft</code>	TD-DFT excited states
<code>g16_read_all</code>	Runs the full extraction set in one call, reading the file only once
<code>g16_restart</code>	Generates a <code>.gjf</code> restart input file from an existing output file
<code>g16_batch_read_all</code>	Runs <code>g16_read_all</code> (+ <code>g16_orbital_energies</code>) over every <code>.log/.out</code> file in a folder and aggregates the key results into one summary DataFrame; per-file failures are recorded, not fatal
<code>g16_draw_molecule</code>	3D CPK ball-and-stick render (matplotlib <code>mplot3d</code>)
<code>g16_draw_mode</code>	3D structure with a vibrational mode’s displacement arrows
<code>g16_draw_orbital</code>	Orbital energy-level diagram with HOMO–LUMO gap arrow
<code>g16_animate_mode</code>	Exports an MP4 animation of a vibrational mode (requires <code>ffmpeg</code> on PATH)
<code>g16_mode_viewer</code>	Interactive Tkinter window to browse/render vibrational modes, sortable by mode number, IR, or Raman intensity, with an “Animate

12.3 Selected signatures

```
g16_read_input(filename)
g16_available_data(filename)
g16_structure(filename, orientation="auto", step="last", lines=None)
g16_energy(filename, step="last", lines=None)
g16_convergence(filename, plot=False, lines=None)
g16_nmodes(filename, section="last", modes=None, lines=None)
g16_spectra(filename, FWHM=10, xmin=0, xmax=4000, dx=1, normalize=False,
            plot=False, section="last", lines=None)
g16_spectra_nm(nm, FWHM=10, xmin=0, xmax=4000, dx=1, normalize=False,
            plot=False)
g16_charges(filename, type="Mulliken", mode="atom", plot=True,
            atom_scale=0.35, bond_tol=1.30, bond_list=None, font_size=8,
            color_scale="RdBu",
            threshold=0.0, show_dipole=False, dipole_origin="negcharge",
            dipole_scale=1.0, dipole_color=(0, 0.6, 0),
            dipole_line_width=2.5,
            show_dipole_label=True, dipole_font_size=11, dipole_units="Debye",
            lines=None)
g16_draw_molecule(mol, atom_scale=0.35, bond_tol=1.30, show_labels=True,
            show_legend=True, title="", bg_color=(0.95, 0.95, 0.95),
            ax=None, show_axes=False, axes_length=None,
            bond_list=None)
g16_animate_mode(mol, nm, mode_idx, filename=None, scale=1.5,
            flip_sign=False,
            atom_scale=0.35, bond_tol=1.30, show_labels=False,
            frames_per_cycle=30, n_cycles=2, fps=20, view=None)
g16_mode_viewer(filename, scale=1.5, atom_scale=0.35, bond_tol=1.30,
            show_labels=False, flip_sign=False, arrow_color=None,
            font_size=10, **extra_opts)
g16_get_bond_length(mol, tolerance=1.15, include_h=True,
            sort_by="distance", save_as="")
g16_nbo_bonds(filename, section="last", lines=None)
g16_fchk_read(filename, verbose=True)
g16_charges_fchk(mol, ch, mode="atom", plot=True, atom_scale=0.35,
            bond_tol=1.30, font_size=8, color_scale="RdBu",
            threshold=0.0)
g16_hyperpolar(filename, units="au", lines=None)
g16_tddft(filename, nstates=math.inf, plot=False, fwhm_ev=0.30, lines=None)
g16_restart(filename, step="last", output="", extra_route="", nproc=0,
            mem="")
g16_batch_read_all(folder, recursive=False, write_reports=False,
            save_as="")
g16_write_report(T, outfile=None)
```

12.4 Notes on the port

Test suite. A pytest suite lives in `G16parser/tests/` (`pip install -e ".[test]"` then `pytest` from the `G16parser/` folder). Most tests need one real Gaussian 16 `.out/.log` file dropped into `tests/fixtures/` (any filename); without one, those tests are skipped rather than failed, so the suite stays runnable out of the box. `tests/fixtures/water.gjf` is a small hand-written synthetic input file (not from a real calculation) used for the `g16_read_input` tests, which do not depend on the optional real-output fixture.

Continuous integration. The test suite runs automatically on every push/pull request to main via GitHub Actions (`.github/workflows/python-tests.yml`), against Python 3.9 and 3.12 on Ubuntu. Coverage is Python-only: running the MATLAB `G09*.m/G16*.m` functions in CI would additionally require a valid MATLAB licence (MathWorks' `matlab-actions/setup-matlab` GitHub Action needs one even for public repositories), so the MATLAB side of the toolbox continues to be verified manually.

3D interactivity. matplotlib's `mplot3d` has no MATLAB-style `rotate3d` widget; use the interactive backend's normal mouse/toolbar controls instead.

Naming exception: `g16_read_input`. On the MATLAB side this function has no version prefix at all (`G_read_input`, identical copies in both the `G09/` and `G16/` folders) because the Gaussian input file format does not differ between versions. Since only a `G16parser` Python package exists (there is no separate `G09parser`), the port simply follows this package's `g16_` naming convention like every other function here.

`g16_restart` and `g16_batch_read_all` port `G16_restart.m` and `G16_batch_read_all.m` only — there is no Python equivalent of `G09_restart.m/G09_batch_read_all.m`, consistent with every other G09-only MATLAB function (e.g. `G09_read_lines`, an internal helper with no G16 counterpart since `G16*.m` functions inline the equivalent file-reading logic instead), since this package only targets Gaussian 16 output. Note that `G09_fchk_read/G09_charges_fchk` are *not* G09-only despite the prefix (identical `G16_` counterparts exist, since the `.fchk` format itself does not depend on the Gaussian version) — they are simply not yet ported to Python either.

Backend selection. The package forces the `TkAgg` matplotlib backend (if Tk is available and `pyplot` has not been imported yet) instead of the native macOS backend: on some older matplotlib releases, interactively rotating a 3D plot with the native Cocoa backend can segfault the whole Python process. Call `matplotlib.use(...)` yourself *before* importing `G16parser` if a different backend is needed.

Bug found during porting, MATLAB-side only: in the original `G16_tddft.m`, MATLAB's `regexp` silently drops the optional trailing `<S**2>=` capture group (a MATLAB-specific regex-engine quirk), so `td.S2` is always `NaN` in MATLAB even when the tag is present in the file. This Python port's `re`-based parser does not have that bug and extracts `S2` correctly. Consider this a known discrepancy between the two implementations rather than a porting error.

Bug found during porting, MATLAB-side only (fixed 2026-08-03): the IR/Raman intensity calculation in `G09_fchk_read.m`/`G16_fchk_read.m` re-shaped the already-correctly-shaped dipole/ polarisability derivative matrices a second time (`reshape(dip_deriv, N3, 3)'`) before use. Since MATLAB's `reshape` reinterprets the same column-major buffer under the new shape rather than transposing back to the original layout, this silently scrambled which derivative value belonged to which atom/displacement, producing plausible-looking but wrong `nm.IR/nm.Raman` values — caught by comparing a `.fchk`-derived spectrum against the same molecule's IR/Raman as printed directly by Gaussian in the corresponding `.out` file. Fixed in both MATLAB files by using the matrices directly (no second reshape needed). This Python port's `g16_fchk_read` never had the bug (written directly with the correct indexing) — verified to match the `.out`-printed values, and a regression test (`test_fchk_ir_raman_matches_out_ground_truth`) now guards against it ever being reintroduced.

`g16_spectra_nm` ports `G_spectra_nm.m` (Section 4.9), a genuinely version-agnostic MATLAB function (no `G09_`/`G16_` prefix, like `G_read_input`): it builds the same Lorentzian-broadened IR/Raman continuum as `g16_spectra`, but directly from an already-parsed `nm` Struct (from `g16_nmodes`, or the `.nm` field of `g16_fchk_read`) instead of reading a `.log`/`.out` file — useful when only a `.fchk` file is available. Verified to match `g16_spectra` called directly on the corresponding `.out` file for the same molecule.

`g16_available_data` ports `G09_available_data.m`/`G16_available_data.m` (Section 10.5). Unlike the MATLAB `G09_` version, it never omits the hyperpolar/`tddft` rows, since this package only targets Gaussian 16 output and both functions always exist here.

Bug fixed (2026-08-05), both MATLAB and Python: `g16_read_all` (and `G09_read_all.m`/`G16_read_all.m`) used to call `g16_nmodes/g16_spectra` unconditionally, so a file with no frequency calculation at all (e.g. a TD-DFT-only single point job) raised `ValueError` and crashed the entire call, even though every other field would have extracted fine. Fixed by wrapping just those two calls in `try/except`: on failure, `nmodes/spectra` are set to `None` (checked via `getattr(T, "nmodes", None)`, matching how `g16_write_report` already consumed them) and a single `warnings.warn(...)` explains why.

`g16_mode_viewer` is a Tkinter rewrite of `G16_modeViewer.m`'s `uifigure` GUI, with the same controls (mode selector, order-by, per-option redraw, title, save-as PDF/EPS/JPEG, and an "Animate mode (MP4)..." button wired to `g16_animate_mode`, using the current option values and the displayed mode figure's camera orientation). One simplification: "target figure" always means the most recently drawn mode window, not whichever window the user last clicked on (MATLAB's `groot().CurrentFigure` has no simple Tkinter equivalent). In an IPython/Spyder console with the Tk graphics backend active, the viewer detects the running event loop and returns to the prompt immediately instead of blocking; run as a plain script, it blocks until the viewer window is closed.

Known issue: `g16_mode_viewer` can segfault when run from *inside* Spyder's IPython console (a crash inside ipykernel's own periodic Tk event-loop pump, not in this toolbox's code) — confirmed working correctly when run from a plain terminal/script instead:

```
python3 -c "import G16parser as g16; g16.g16_mode_viewer('file.out')"
```

All other functions, including static plots, work fine inside Spyder.

g16_animate_mode mirrors `G09_animate_mode.m/ G16_animate_mode.m`: it oscillates the molecule along the mode's displacement vector and saves an MP4 via `matplotlib.animation.FuncAnimation` with `FFMpegWriter`. **Requires ffmpeg installed and on PATH** (brew install ffmpeg on macOS, `sudo apt install ffmpeg` on Ubuntu/Debian) — `matplotlib` does not bundle a video encoder itself; without it the call fails with `FileNotFoundError: ... 'ffmpeg' at the final encoding step` (frame generation itself does not need ffmpeg). Bonds are computed once from the equilibrium geometry via `g16_get_bond_length` and passed to `g16_draw_molecule` through its `bond_list` parameter, so they stay fixed across all frames instead of flickering as instantaneous distances cross `bond_tol`. By default the animation uses `matplotlib`'s default 3D view; pass `view=(ax.azim, ax.elev)` to start from an already-rotated figure's orientation instead.

g16_write_report takes the `Struct` returned by `g16_read_all` and writes the same section-by-section plain-text report as `G09_write_report.m/G16_write_report.m` (route, charge/multiplicity, structure, energetics, dipole/polarisability, atomic charges, normal modes, spectra summary), using Python `getattr`-based field checks so that sections are included only when present, exactly mirroring the MATLAB `isfield` logic.

Bond order (single/double/triple). `g16_draw_molecule` mirrors `G09_draw_molecule.m/G16_draw_molecule.m`: for C-C and C-O pairs, bond order is estimated purely from bond length (any other element pair, including C-N, is always drawn as a single bond) and rendered as 1/2/3 parallel lines. This is a geometric estimate, not Gaussian's own bond-order analysis (e.g. Wiberg/NBO indices). For an actual bond order derived from Gaussian's own NBO analysis, use `g16_nbo_bonds` (Section 10.2). The `bond_list` parameter accepts an optional 3rd column with a pre-computed order, used by `g16_animate_mode` (via `g16_get_bond_length`) to keep both bond topology and bond order fixed across all animation frames:

```
mol = g16.g16_structure('molecule_nbo.out')
bt  = g16.g16_nbo_bonds('molecule_nbo.out')    # requires 'pop=nbo'
      in the source job

bond_list = bt[['Atom1', 'Atom2', 'BondOrder']].to_numpy()
bond_list[:, :2] -= 1    # g16_nbo_bonds uses 1-based Gaussian atom
                        # g16_draw_molecule's bond_list is 0-based
                        # g16_draw_molecule's bond_list is 0-based
g16.g16_draw_molecule(mol, bond_list=bond_list)
```

`g16_charges` accepts the same `bond_list=` parameter, passed straight through to its internal `g16_draw_molecule` call:

```
g16.g16_charges('molecule_nbo.out', bond_list=bond_list,
                show_dipole=True)
```

The C-C double/single threshold is set to 1.36 Å rather than the generic reference-length midpoint, so symmetric aromatic rings (C-C $\approx$ 1.39–1.40 Å, no real length alternation) render as all-single instead of all-double. C-N is deliberately excluded from length-based classification: on asymmetric pyridine-like rings a single threshold produced one ring C-N bond rendered double and its chemically-equivalent neighbour rendered single — see the

MATLAB `G09_draw_molecule/G16_draw_molecule` subsection earlier in this manual for the full rationale.

13 Worked examples

Four self-contained scripts in G16/ (`example_1.m`, `example_2.m`, `example_3.m`, `example_4.m`) demonstrate the toolbox end to end on a single real DFT calculation: `test_2.out`, a geometry optimisation of Violacein (G. Cassone et al., *Phys. Chem. Chem. Phys.*, doi: 10.1039/d6cp01164k). Each script only needs G16/ on the MATLAB path and `test_2.out` in the current folder.

13.1 `example_1.m` — Structure, vibrational mode, and charges

Covers `G16_structure`/`G16_draw_molecule` (3D structure rendering), `G16_nmodes`/`G16_draw_mode` (a single vibrational mode, #91, the main C=C stretch), and `G16_charges` (Mulliken charges with a dipole moment overlay), all rendered from the same camera angle and each exported to its own PDF.

```
%% G16 Toolbox -- Example #1: structure, vibrational mode, and charges
%
%   Demonstrates three core G16_*.m functions on a real DFT
%   calculation:
%       1) G16_structure   + G16_draw_molecule   -- 3D structure rendering
%       2) G16_nmodes     + G16_draw_mode        -- a single vibrational
%           mode
%       3) G16_charges          -- Mulliken charges +
%           dipole
%
%   Data file: test_2.out -- DFT geometry optimisation of Violacein.
%   Reference: G. Cassone et al., Phys. Chem. Chem. Phys.,
%               doi: 10.1039/d6cp01164k
%
%   Requirements: G16/ on the MATLAB path, test_2.out in the current
%   folder. Running this script creates three PDFs in the current
%   folder: test_2_mol.pdf, test_2_mode_91.pdf, test_2_charges.pdf.

clear
close all
clc

filename = 'test_2.out';

% Camera orientation shared by all three figures below, so the
%   molecule
% is viewed from the same angle in every plot.
view_az = -16.4417;
view_el = 44.0197;

%% 1) Structure -- load the optimised geometry and render it in 3D
mol = G16_structure(filename);

% Create the axes up front so it can be tweaked (view angle, export)
% after G16_draw_molecule has finished drawing into it.
ax1 = axes;
set(ax1, 'Visible', 'off');
G16_draw_molecule(mol, 'Title', 'Violacein', 'Ax', ax1);
set(ax1, 'View', [view_az, view_el]);

% ContentType 'vector' gives a crisp publication figure but is slow
%   for
% a molecule this size; use 'image' instead for a quick low-effort
%   preview.
```

```

exportgraphics(ax1, 'test_2_mol.pdf', 'ContentType', 'vector');

%% 2) Vibrational mode -- mode #91 is the main C=C stretch
nm = G16_nmodes(filename);

ax2 = axes;
set(ax2, 'Visible', 'off');
G16_draw_mode(mol, nm, 91);
set(ax2, 'View', [view_az, view_el]);

% Replace only the molecule-name line of the auto-generated two-line
% title; the second line (mode number, frequency, IR/Raman intensity)
% is left untouched.
t = get(gca, 'Title');
t.String(1) = {'Violacein'};
set(gca, 'Title', t);

exportgraphics(gca, 'test_2_mode_91.pdf', 'ContentType', 'vector');

%% 3) Atomic charges -- Mulliken charges with a dipole moment overlay
G16_charges(filename, 'Plot', true, 'ShowDipole', true);
set(gca, 'View', [view_az, view_el]);

t = get(gca, 'Title');
t.String = {'Violacein - Mulliken Charges (atom)'};
set(gca, 'Title', t);

exportgraphics(gca, 'test_2_charges.pdf', 'ContentType', 'vector');

fprintf('\nDone. Figures: structure, mode 91, Mulliken charges.\n');
fprintf('Saved: test_2_mol.pdf, test_2_mode_91.pdf, test_2_charges.pdf\n');

```

13.2 example_2.m — Infrared and Raman spectra

Demonstrates G16_spectra two ways: a manual plot built directly from its raw `sp.x/sp.Raman_cont` output fields, and the function's own built-in two-panel Raman+IR auto-plot (`'plot', true`), restyled afterwards with larger, bold axis labels for a print-ready figure. Both are exported to PDF.

```

%% G16 Toolbox -- Example #2: Infrared and Raman spectra
%
%   Demonstrates G16_spectra on a real DFT calculation, two ways:
%   1) Manual plot -- build a single Raman continuum plot directly
%   from the raw sp.x/sp.Raman_cont fields returned by
%   G16_spectra.
%   2) Built-in plot -- let G16_spectra draw its own two-panel
%   Raman+IR figure ('plot', true), then restyle the axes/labels
%   afterwards (larger, bold text) for a print-ready figure.
%
%   Data file: test_2.out -- DFT geometry optimisation of Violacein.
%   Reference: G. Cassone et al., Phys. Chem. Chem. Phys.,
%   doi: 10.1039/d6cp01164k
%
%   Requirements: G16/ on the MATLAB path, test_2.out in the current
%   folder. Running this script creates two PDFs in the current
%   folder: test_2_raman_manual.pdf, test_2_ir_raman.pdf.

```

```

clear; close all; clc
filename = 'test_2.out';
%% 1) Manual plot -- Raman continuum only, from the raw sp fields
sp = G16_spectra(filename);
fig1 = figure('Color', 'white');
plot(sp.x, sp.Raman_cont, 'LineWidth', 2);
xlabel('Wavenumber (cm-1)', 'FontWeight', 'bold');
ylabel('Raman activity (A4 AMU-1)', 'FontWeight', 'bold');
set(gca, 'FontSize', 12);
exportgraphics(fig1, 'test_2_raman_manual.pdf', 'ContentType', 'vector');
%% 2) Built-in two-panel plot (Raman + IR), then restyled
% 'FWHM' broadens the Lorentzian lines and 'xmin' crops the noisy
% low-frequency tail; 'plot', true triggers G16_spectra's own
% two-subplot Raman/IR figure (see G16_plot_spectra, local to
% G16_spectra.m).
G16_spectra(filename, 'FWHM', 15, 'xmin', 400, 'plot', true);
fig2 = gcf;
% G16_spectra creates the Raman axes first, then the IR axes below it;
% findobj returns them in reverse creation order, so flip to restore
% [Raman; IR].
axs = flipud(findobj(fig2, 'Type', 'axes'));
ax_raman = axs(1);
ax_ir = axs(2);
% Upsize the auto-generated labels (FontSize 10, regular weight) to a
% bolder, larger style for a print-ready figure.
set([ax_raman, ax_ir], 'FontSize', 12);
xlabel(ax_raman, 'Wavenumber (cm-1)', 'FontWeight', 'bold',
       'FontSize', 12);
ylabel(ax_raman, 'Raman activity (A4 AMU-1)', 'FontWeight', 'bold',
       'FontSize', 12);
xlabel(ax_ir, 'Wavenumber (cm-1)', 'FontWeight', 'bold',
       'FontSize', 12);
ylabel(ax_ir, 'IR intensity (KM mol-1)', 'FontWeight', 'bold',
       'FontSize', 12);
exportgraphics(fig2, 'test_2_ir_raman.pdf', 'ContentType', 'vector');
fprintf('\nDone. Figures: manual Raman plot, built-in IR+Raman plot.\n');
fprintf('Saved: test_2_raman_manual.pdf, test_2_ir_raman.pdf\n');

```

13.3 example_3.m — Energetics

Demonstrates `G16_orbital_energies`, `G16_energy`, and `G16_convergence`, renders the HOMO-LUMO diagram via `G16_draw_orbital`, and combines all three structs into a one-line summary — a small-scale taste of what `G16_read_all`/`G16_write_report` do at full scale across every extraction function.

```

%% G16 Toolbox -- Example #3: Energetics
%
% Demonstrates G16_orbital_energies, G16_energy, and G16_convergence
% on a real DFT calculation, then combines their results into a
% one-line summary -- a small taste of what G16_read_all/
% G16_write_report do at full scale across every extraction function
%
% Also renders the HOMO-LUMO diagram via G16_draw_orbital.
%

```

```

% Data file: test_2.out -- DFT geometry optimisation of Violacein.
% Reference: G. Cassone et al., Phys. Chem. Chem. Phys.,
% doi: 10.1039/d6cp01164k
%
% Requirements: G16/ on the MATLAB path, test_2.out in the current
% folder. Each function already prints its own formatted summary to
% the command window (shown below as a reference for what to expect
% on this file); this script does not repeat those printouts.
Running
% it creates test_2_orbital_diagram.pdf in the current folder.

clear; close all; clc
filename = 'test_2.out';
%% 1) HOMO-LUMO gap -- detailed data in the oe structure
oe = G16_orbital_energies(filename);
% Console output for this file:
% -- G16_orbital_energies (step 3/3): test_2.out --
% HOMO = -0.190210 Ha (-5.1759 eV)
% LUMO = -0.096890 Ha (-2.6365 eV)
% Gap = 0.093320 Ha (2.5394 eV)

% Orbital energy-level diagram (title defaults to the source filename)
G16_draw_orbital(oe);
exportgraphics(gcf, 'test_2_orbital_diagram.pdf', 'ContentType', 'vector');
%% 2) Energy and thermochemistry -- detailed data in the en structure
en = G16_energy(filename);
% Console output for this file:
% -- G16_energy: test_2.out --
% Method : RB3LYP
% SCF : -1160.20276011 Ha
% ZPE : +0.29182000 Ha (766.17 kJ/mol)
% E0+ZPE : -1159.91094000 Ha
% H : -1159.88988100 Ha
% G : -1159.96045200 Ha
% S : 621.4460 J/(mol.K)
% T = 298.15 K, P = 1.00000 atm

%% 3) Geometry-optimisation convergence -- detailed data in the cv
structure
cv = G16_convergence(filename);
% Console output for this file:
% G16_convergence: test_2.out
% Steps read : 11
% Converged : YES (step 10)
% Last step : MaxF=1.00e-06 (thr 1.50e-05) RMSF=0.00e+00 (thr
1.00e-05)
% MaxD=1.24e-04 (thr 6.00e-05) RMSD=2.50e-05 (thr
4.00e-05)

%% 4) Combine all three into a one-line summary
if cv.converged
conv_label = sprintf('YES (step %d/%d)', cv.conv_step, cv.Nsteps);
else
conv_label = sprintf('NO (%d steps read)', cv.Nsteps);
end
fprintf('\nSummary for %s:\n', filename);
fprintf(' HOMO-LUMO gap : %.3f eV\n', oe.gap_eV);

```

```

if en.has_thermo
    fprintf('  Gibbs energy   : %.2f kJ/mol\n', en.G_kJ);
else
    fprintf('  Gibbs energy   : n/a (no frequency/thermochemistry job)\n');
end
fprintf('  Converged          : %s\n', conv_label);

```

13.4 example_4.m — Recovering and restyling graphics handles

Shows how to reach back into a figure already drawn by `G16_draw_molecule` and restyle every part of it after the fact (title, axis labels, legend, atom spheres, bond lines, atom-label text), rather than only being able to set style options at call time.

The key gotcha this example exists to document: most of the graphics objects `G16_draw_molecule` creates (all bond lines, and every atom sphere after the first drawn for a given element) are created with `'HandleVisibility', 'off'`, so they do not clutter the legend or turn up in a plain `findobj` search. Reaching them requires `findall` instead of `findobj`. Title, axis labels, and the legend itself keep the default (`'on'`) visibility, so either function finds those. The script also re-enables the axes (`axis(ax, 'on')`) to reveal the X/Y/Z labels, which `G16_draw_molecule` always creates but hides by calling `axis(ax, 'off')` internally; and separates the atom-label text objects from the title/axis-label text objects (all four share `Type = 'text'`) with a `setdiff`.

```

%% Example 4
%% Recover graphics handles from G16_draw_molecule and restyle them
mol = G16_structure('test_2.out');
G16_draw_molecule(mol, 'Title', 'Violacein');

ax = gca;
fig = gcf;

%% 1) Title -- always visible (HandleVisibility default 'on')
title(ax, 'Violacein (DFT-optimised)', 'FontSize', 13, ...
      'FontWeight', 'bold', 'Interpreter', 'tex');

%% 2) Axis labels -- G16_draw_molecule calls axis(ax,'off'), so labels
%    exist but are not shown until the axis is switched back on
axis(ax, 'on');
xlabel(ax, 'X (\AA)', 'FontSize', 11, 'Interpreter', 'latex');
ylabel(ax, 'Y (\AA)', 'FontSize', 11, 'Interpreter', 'latex');
zlabel(ax, 'Z (\AA)', 'FontSize', 11, 'Interpreter', 'latex');

%% 3) Legend -- created with default HandleVisibility, findobj works
fine
hleg = findobj(fig, 'Type', 'Legend');
set(hleg, 'FontSize', 10, 'Location', 'northeast', 'Box', 'on');

%% 4) Atom spheres (Surface objects) -- use findall: most have
%    HandleVisibility 'off' (only the first atom per element is 'on')
hatoms = findall(ax, 'Type', 'Surface');
set(hatoms, 'FaceAlpha', 0.9, 'EdgeColor', 'none');

%% 5) Bond lines -- all created with HandleVisibility 'off', findall
%    required
hbonds = findall(ax, 'Type', 'Line');

```

```

set(hbonds, 'LineWidth', 2.5, 'Color', [0.35 0.35 0.35]);

%% 6) Atom-label text objects only, excluding Title/XLabel/YLabel/
    ZLabel
%    (these are also Type='text', so must be explicitly subtracted out
    )
htitle_h = get(ax, 'Title');
hxlabb    = get(ax, 'XLabel');
hyllabb    = get(ax, 'YLabel');
hzllabb    = get(ax, 'ZLabel');
halltext = findall(ax, 'Type', 'Text');
hatomlbl = setdiff(halltext, [htitle_h; hxlabb; hyllabb; hzllabb]);
set(hatomlbl, 'FontSize', 8, 'FontWeight', 'bold');

%% 7) Background / figure colour
set(ax, 'Color', [1 1 1]);
set(fig, 'Color', [1 1 1]);

exportgraphics(fig, 'test_2_mol_restyled.pdf', 'ContentType', 'vector
    ');

```

14 Publication and licensing

14.1 Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the author(s) used Claude (Anthropic) to assist with code drafting, refactoring, and documentation of the G09/G16 toolbox and its associated user manual. After using this tool, the author(s) reviewed and edited the content as needed and take full responsibility for the content of the publication.

14.2 Note on the Use of AI-Assisted Development Tools

Portions of the codebase and documentation described in this manual — including the G09_/G16_ MATLAB function pairs, the `g16toolbox` Python package, the `G09_modeViewer.m`/ `G16_modeViewer.m` interactive GUIs, and this reference manual — were developed with the assistance of Claude (Anthropic), an AI-based coding tool, used interactively under the direct supervision of the author. All AI-generated code was tested against reference Gaussian 09/16 output files and reviewed by the author prior to inclusion. Algorithm design, scientific validation, and correctness of results remain the sole responsibility of the author.

14.3 Licence

This software is released under the **BSD 3-Clause License**, an OSI-approved open source license accepted by SoftwareX. See the `LICENSE.txt` file distributed with the source code for the full licence text.

14.4 Citation

If this toolbox contributes to published research, please acknowledge it by citing the source repository:

S. Trusso, *G09/G16 Toolbox: MATLAB and Python libraries for parsing, analysing and visualising Gaussian 09/16 output files*, https://github.com/nello62/Gaussian09G16_output_Matlab_parser (accessed August 6, 2026).